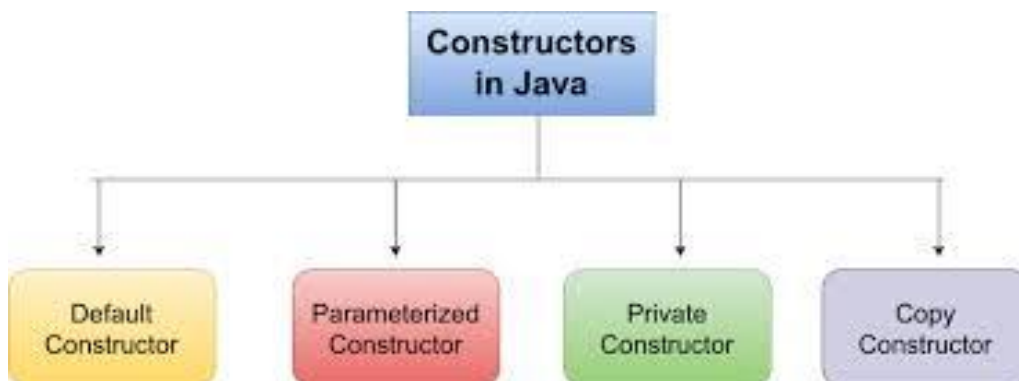


OOPS Concept

Constructors in Java :-

- A constructor is a special method used to initialize objects in Java.
- The name of the constructor is same as the class name.
- It does not have a return type, not even void.
- It is called automatically at the time of object creation.
- If no constructor is defined, Java provides a default constructor.



2. Parameterized Constructor :-

- A constructor that takes parameters while creating an object.
- It is used to initialize objects with given values.
- Provides flexibility by passing different values for each object.
- Called automatically when arguments are passed at object creation.

Syntax –

```
class ClassName {  
    // parameterized constructor  
    ClassName(datatype var1, datatype var2, ...) {  
        // initialization code  
    }  
}
```

Example –

```
package constructor;

public class parameterized_constructor {
    parameterized_constructor(int a,int b, int c) //parameterized constructor
    {
        System.out.println("Multiplication is: "+(a*b*c));
    }

    parameterized_constructor(int a,int b) //parameterized constructor
    {
        System.out.println("Division is: "+(a/b));
    }

    parameterized_constructor(byte a,byte b,byte c) //parameterized constructor
    {
        System.out.println("Addition is: "+(a+b+c));
    }

    parameterized_constructor(byte a,byte b) //parameterized constructor
    {
        System.out.println("Substraction is: "+(a-b));
    }

    public static void main(String[] args) {
        //calling parameterized constructor
        parameterized_constructor p=new parameterized_constructor(2,3,4);
        parameterized_constructor p1=new parameterized_constructor(6,2);
        parameterized_constructor p2=new parameterized_constructor((byte)4,(byte)5,(byte)8);
        parameterized_constructor p3=new parameterized_constructor((byte)7,(byte)5);
    }
}
```

Output -

```
Multiplication is: 24
Division is: 3
Addition is: 17
Substraction is: 2
```

3. Private Constructor :-

- A constructor declared with the private access modifier.
- It cannot be accessed outside the class directly.
- Used to restrict object creation from other classes.
- Commonly used in Singleton Design Pattern and utility classes.

Syntax –

```
class ClassName {  
    private ClassName() {  
        // initialization code  
    }  
}
```

Example –

```
package constructor;  
  
public class private_constructor {  
    private_constructor()  
    {  
        // private constructor  
        System.out.println("Private Constructor");  
    }  
  
    public static void main(String[] args) {  
        //calling private_constructor  
        private_constructor t = new private_constructor();  
    }  
}
```

Output –

Private Constructor

4. Copy Constructor :-

- A copy constructor is a constructor that creates a new object by copying values from another object.
- It is not provided by default in Java (we create it manually).
- It takes an object of the same class as a parameter.
- Useful for creating a duplicate object with the same data.

Syntax –

```
class ClassName {  
    ClassName(ClassName obj) {  
        // copy values from obj to this object  
    }  
}
```

Example –

```
package constructor;  
  
public class employee {  
    //Global Variable/ Instance Variable  
    int emp_id;  
    String emp_name;  
    employee(int emp_id,String emp_name) //parameterized constructor  
    {  
        this.emp_id=emp_id;  
        this.emp_name=emp_name;  
        System.out.println("Employee_Id: "+this.emp_id);  
        System.out.println("Employee_Name: "+this.emp_name);  
    }  
  
    employee(employee e) //copy constructor  
    {  
        this.emp_id=e.emp_id;  
        this.emp_name=e.emp_name;  
        System.out.println("Copy of Employee_Id is: "+this.emp_id);  
        System.out.println("Copy of Employee_Name is: "+this.emp_name);  
    }  
  
    public static void main(String[] args) {  
        employee emp1=new employee(25,"Sagar"); //calling parameterized constructor  
  
        employee emp2=new employee(25,"Sagar"); //calling copy constructor  
    }  
}
```

Output –

Employee_Id: 25
Employee_Name: Sagar
Employee_Id: 25
Employee_Name: Sagar

Task –

```
package practical;

public class student {
    int s_rollno;
    String s_name;
    String s_city;
    long s_phno;

    student(int s_rollno,String s_name,String s_city,long s_phno)
    {
        this.s_rollno=s_rollno;
        this.s_name=s_name;
        this.s_city=s_city;
        this.s_phno=s_phno;

        System.out.println("Student_RollNo is: "+this.s_rollno);
        System.out.println("Student_Name is: "+this.s_name);
        System.out.println("Student_City is: "+this.s_city);
        System.out.println("Student_PhNo is: "+this.s_phno);
    }

    student(student st)
    {
        this.s_rollno=st.s_rollno;
        this.s_name=st.s_name;
        this.s_city=st.s_city;
        this.s_phno=st.s_phno;

        System.out.println("Copy of Student_RollNo is: "+this.s_rollno);
        System.out.println("Copy of Student_Name is: "+this.s_name);
        System.out.println("Copy of Student_City is: "+this.s_city);
        System.out.println("Copy of Student_PhNo is: "+this.s_phno);
    }

    public static void main(String[] args) {
        student s=new student(101,"Sagar","Satara",9890824905L);
        student s1=new student(s);
    }
}
```

Output –

Student_RollNo is: 101
Student_Name is: Sagar
Student_City is: Satara
Student_PhNo is: 9890824905
Copy of Student_RollNo is: 101
Copy of Student_Name is: Sagar
Copy of Student_City is: Satara
Copy of Student_PhNo is: 9890824905